



Prof. Dr. Bernhard Seeger
Manuel Oed, M.Sc.

Programmierpraktikum

AG Datenbanksysteme

Aufgabe 1: Einarbeitung in die Java Streaming API

(4 Punkte)

Stellen Sie sicher, dass Sie die Java-Version 25 verwenden. Sie können diese **hier** oder über Ihre Arbeitsumgebung **IntelliJ IDEA** automatisch herunterladen (empfohlen). Arbeiten Sie sich für folgende Aufgaben in die `Stream`-API von Java 25 ein. Lesen Sie dazu folgende Dokumentationen durch und machen Sie sich mit den Funktionalitäten vertraut:

- Das `Stream` Interface.
- Die `Optional` Klasse.
- Die `Collectors` Klasse. item Das `Comparator` Interface.

Schauen Sie sich auch die in der Dokumentation beschriebenen Beispiele an. Ihr Tutor wird Ihnen zu jedem Punkt drei Fragen stellen, um sicherzustellen, dass Sie die Grundlagen kennen. Eine Frage könnte z.B. sein: Was ist ein `IntStream`?

Weiter ist für die Bearbeitung der Aufgaben Folgendes zu beachten:

1. Sie können für Ihre Lösungen beliebige Methoden verwenden, jedoch müssen die in einem **Hinweis** genannten Methoden in der Lösung der jeweiligen Teilaufgabe mindestens einmal vorkommen.
2. Laden Sie sich das Klassen-Gerüst aus dem Ilias herunter und bearbeiten Sie darin die folgenden Aufgaben.
3. Es gibt insgesamt 30 Punkte heute. Davon müssen Sie mindestens **15** Punkte sammeln, um zu bestehen.

Viel Erfolg bei der Bearbeitung!

Aufgabe 2: Einführung in die Java Streaming API (1+1+1+2+2+1)

(8 Punkte)

Bearbeiten Sie folgende Aufgaben in der Klasse `StreamingJava`. Verlassen Sie für die Bearbeitung dieser Teilaufgaben **nicht** die `Stream`-API.

- a) Vervollständigen Sie die Methode `<E> Stream<E> flatStreamOf(List<List<E>> list)`, um die Eingabe `list` zu einem `Stream<E>` zu transformieren.

Hinweis: Nutzen Sie die `Stream`-Methode `flatMap`.

- b) Vervollständigen Sie die Methode `int bitsOf(IntStream stream)`, die alle Werte aus `stream` bitweise (ODER) zu einer einzigen Maske zusammenfasst. Für einen leeren `stream` soll 0 zurückgegeben werden.

Hinweis: Nutzen Sie die `Stream`-Methode `reduce`.

- c) Vervollständigen Sie die Methode `<E extends Comparable<? super E>> E minOf(List<List<E>> list)`, um aus der übergebenen `list` das Minimum zu ermitteln. Nutzen Sie einen parallelen `Stream` und für die Ordnung die natürliche Ordnung der Elemente. Gibt es kein Minimum, soll eine Exception geworfen werden.

Hinweis: Nutzen Sie die `Stream` Methode `min` und die `Optional`-Methode `orElseThrow()`.

- d) Vervollständigen Sie die Methode `<E> E lastWithOf(Stream<E> stream, Predicate<? super E> predicate)`, um das letzte Element zu finden, welches das Prädikat `predicate` erfüllt.

Hinweis: Nutzen Sie die `Stream` Methode `filter`.

- e) Vervollständigen Sie die Methode `<E> Set<E> findOfCount(Stream<E> stream, int count)`, um ein `Set<E>` zurückzugeben, das alle Elemente aus `stream` enthält, die genau `count` oft in `stream` vorkommen.

Hinweis: Nutzen Sie die `Stream Collector`-Methode `Collectors.groupingBy()`.

- f) Vervollständigen Sie die Methode `IntStream makeStreamOf(String[] strings)`, um das übergebene `String`-Array in einen `IntStream` zu transformieren. Dabei sollen die einzelnen Buchstaben der Zeichenketten in dem zurückgegebenen `Stream` enthalten sein.

Hinweis: Nutzen Sie die `Stream`-Methode `Stream.flatMapToInt()`.

Aufgabe 3: Datenverarbeitung (2+1+1+3+1+3+3+4)

(18 Punkte)

In dieser Aufgabe sollen Sie Daten aus einer **CSV**-Datei verarbeiten. Um bestens für die Bearbeitung folgender Aufgaben vorbereitet zu sein, machen Sie sich zuerst mit folgenden Funktionalitäten aus dem **aus den Paketen java.nio.file** und **java.io** vertraut:

1. Die Klasse `Files`.
2. Das Interface `Path`.
3. Die Klasse `InputStream`.
4. Die Klasse `OutputStream`.
5. Das `try-with-resources` Statement.

Bearbeiten Sie folgende Teilaufgaben in der Klasse `StreamingJava`. Weiter sollen Sie die Datei namens **NaturalGasBilling.csv** aus der Vorlage laden können. Dazu sollen Sie Ihr Projektverzeichnis kennen, um in den Teilaufgaben auf diese Datei zugreifen zu können.

- a) Vervollständigen Sie die Methode `Stream<String> fileLines(String path)`, um die Datei namens **NaturalGasBilling.csv** zu laden. Dabei sollen Sie einen **BufferedReader** nutzen und Zeilen des Inhalts über den `Stream<String>` zurückgeben. Achten Sie dabei darauf, dass die erste Zeile übersprungen wird und der zurückgegebene `Stream<String>` sequentiell ist. Sobald der zurückgegebene `Stream<String>` geschlossen wird, soll auch der **BufferedReader** geschlossen werden und eine Konsolen-Ausgabe soll über die erfolgreiche Schließung des Readers berichten.

Hinweis: Nutzen Sie die Methode `Files.newBufferedReader()`.

- b) Vervollständigen Sie die Methode `double averageCost(Stream<String> lines)`, um die durchschnittlichen Kosten (Spalte **Amount**) aus allen Einträgen aus `lines` (Übergabe aus a)) zu berechnen.

Hinweis: Nutzen Sie die Stream-Methode `DoubleStream.average()`.

- c) Vervollständigen Sie die Methode `long countCleanEnergyLevy(Stream<String> lines)`, um die Einträge `lines` (Übergabe aus a)) zu zählen, die keinen (leer oder 0) `Clean energy levy` Betrag gezahlt haben.

Hinweis: Nutzen Sie die Stream-Methode `Stream.count()`.

- d) Erstellen Sie einen record namens `NaturalGasBilling`, um Datensätze aus der Datei **NaturalGasBilling.csv** darzustellen. Dazu sollen Sie jedes Attribut als Feld definieren. Weiter sollen Sie die Methode `Stream<NaturalGasBilling> orderByInvoiceDateDesc(Stream<String> stream)` erzeugen, um aus dem übergebenen `stream` einen nach **Invoice Date** absteigend sortierten `Stream<NaturalGasBilling>` zu erhalten.

Hinweis: Nutzen Sie die Comparator-Methode `Comparator.comparing()`.

- e) Fügen Sie dem record `NaturalGasBilling` die Methode `String toCsvLine()` hinzu, die den Datensatz in eine **CSV**-Zeile transformiert und dabei das Format bzw. die Reihenfolge der **CSV**-Datei **NaturalGasBilling.csv** verfolgt. Das Zeilenende (`\n`) gehört nicht dazu; es wird erst in f) gesetzt.

Hinweis: Nutzen Sie die Methode `String.join()`.

- f) Erstellen Sie die Methode `void serialize(Stream<NaturalGasBilling> stream, OutputStream out)`, die den übergebenen `stream` als **CSV** über den `OutputStream out` schreibt. Achten Sie dabei darauf, dass die erste Zeile die Attribute beinhaltet, gefolgt von den serialisierten Datensätzen. Der `OutputStream` soll innerhalb der Methode nicht geschlossen werden; stellen Sie aber sicher, dass alle Daten vollständig hinausgeschrieben werden. Nutzen Sie nun in der `main`-Methode Ihre `serialize()` und `orderByInvoiceDateDesc()` Methoden und speichern Sie die Datensätze aus `NaturalGasBilling.csv` absteigend sortiert nach `Invoice Date` in eine neue CSV-Datei. Stellen Sie sicher, dass danach alles korrekt geschlossen wird.

Hinweis: Nutzen Sie die `Stream.concat()` Methode für `serialize()`.

- g) Erzeugen Sie die Methode `Stream<NaturalGasBilling> deserialize(InputStream in)`, um die in f) geschriebene **CSV**-Datei über den `InputStream in` zurück in einen `Stream<NaturalGasBilling>` zu überführen. Der zurückgegebene `Stream` soll *lazy* sein, d.h. es soll immer nur so viel aus `in` gelesen werden, wie für das nächste Element benötigt wird. Öffnen Sie nun in der `main`-Methode die Datei aus f) als `InputStream`, rufen Sie damit `deserialize()` auf und schreiben Sie das Resultat auf die Konsole. Stellen Sie sicher, dass danach alles korrekt geschlossen wird.

Hinweis: Nutzen Sie die Methode `BufferedReader.lines()` sowie `Stream.onClose()`, um das Schließen des Readers in dem zurückgegebenen `Stream` zu registrieren.

- h) Vervollständigen Sie die Methode `Stream<File> findFilesWith(String dir, String st, String ed, int maxFiles)`, um das Verzeichnis `dir` nach Dateien zu durchsuchen. Es sollen alle Dateien (auch in Unterverzeichnissen), die mit `st` beginnen und mit `ed` enden, gefunden werden. Weiterhin sollen die gefundenen Dateien nach Größe absteigend sortiert zurückgegeben werden. Es dürfen dabei maximal `maxFiles` viele Dateien zurückgegeben werden. Machen Sie in der `main`-Methode einen Aufruf Ihrer Methode, um alle Java-Source-Code-Dateien aus Ihrem Java-Projekt zurückzubekommen. Stellen Sie sicher, dass danach alles korrekt geschlossen wird.

Hinweis: Nutzen Sie die Methode `Files.walk()`.